DATA LEAKAGE PREVENTION AND DETECTION SYSTEM

Project Documentation

1. Abstract

Data leakage is one of the most critical cybersecurity threats faced by organizations today. Sensitive information such as API keys, passwords, access tokens, database credentials, and confidential organizational data are frequently exposed due to human error, insecure coding practices, or improper handling of information. Such incidents can result in unauthorized access, financial losses, compliance violations, and reputational damage.

The Data Leakage Prevention and Detection (DLP) System is designed to proactively identify and prevent the exposure of sensitive information before it leaves the development environment. The solution combines a web-based monitoring platform and a Visual Studio Code extension. The VS Code extension performs real-time scanning of source code files and alerts developers when sensitive information is detected. The web application provides centralized monitoring, incident management, analytics, and reporting capabilities.

The system enhances organizational security by helping developers and administrators identify security risks early, reduce accidental exposure of confidential data, and improve secure coding practices.

2. Introduction

As software development becomes increasingly collaborative and cloud-based, developers frequently work with API keys, passwords, authentication tokens, and confidential organizational information. In many cases, these secrets are unintentionally embedded within source code and later exposed through public repositories, internal sharing platforms, or deployment pipelines.

Traditional security tools often detect data leaks only after information has already been exposed. This reactive approach increases the likelihood of security incidents and makes remediation more difficult.

The Data Leakage Prevention and Detection System addresses this challenge by providing real-time detection and prevention mechanisms. By integrating directly into the developer workflow through a VS Code extension and providing centralized visibility through a web dashboard, the system helps organizations strengthen their security posture and reduce data leakage risks.

3. Problem Statement

Organizations face significant challenges in preventing sensitive information leakage during software development and data handling processes.

Common issues include:

- Hardcoded API keys in source code
- Exposed database credentials
- Leaked authentication tokens
- Accidental disclosure of confidential information
- Lack of centralized monitoring
- Delayed detection of security incidents

Existing solutions often identify leaks only after data has been exposed, increasing the impact of security incidents.

Therefore, there is a need for a proactive solution capable of detecting and preventing sensitive information exposure before it becomes a security threat.

4. Objectives

Primary Objectives

- Detect sensitive information in source code files.
- Prevent accidental data exposure.
- Generate real-time security alerts.
- Maintain centralized incident records.
- Provide monitoring and reporting capabilities.

Secondary Objectives

- Improve developer security awareness.
- Strengthen secure coding practices.
- Reduce organizational security risks.
- Enable proactive threat management.

5. Scope of the Project

The project focuses on detecting sensitive information during software development and providing centralized monitoring capabilities.

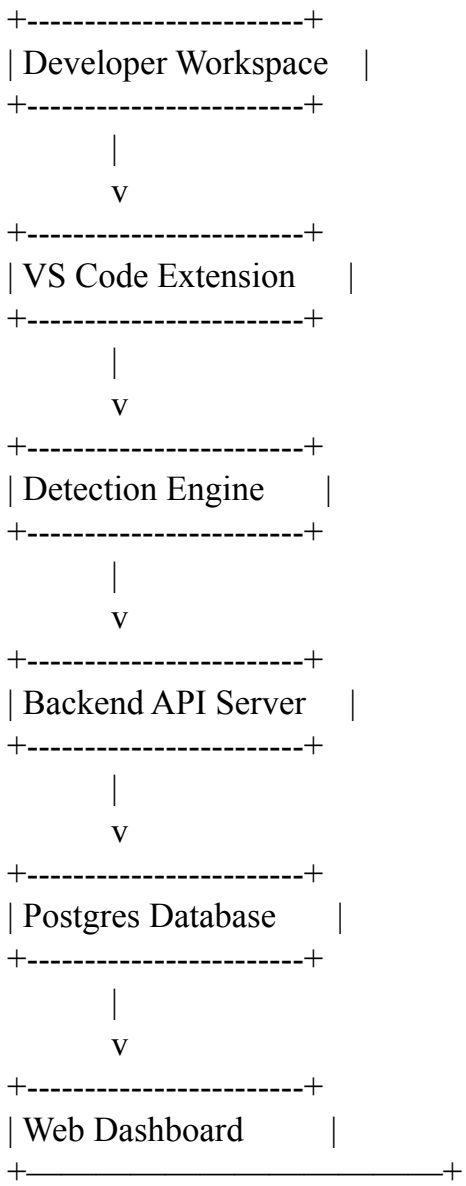
Included Scope

- Source code scanning
- Secret detection
- Incident logging
- Alert generation
- Dashboard visualization
- User authentication
- Incident management

Future Scope

- AI-based detection
- GitHub repository scanning
- Cloud deployment
- Automated remediation
- Compliance monitoring

6. System Architecture



Architecture Description

The VS Code extension continuously monitors files during development. When sensitive information is detected, the detection engine generates alerts and forwards incident information to the backend server. The backend stores incident data in the database and makes it available through the web dashboard for administrators to review and manage.

7. Technology Stack

Frontend

- React.js
- HTML5
- CSS3
- JavaScript

Backend

- Node.js
- Express.js

Database

- MongoDB

Extension Development

- VS Code Extension API
- JavaScript / TypeScript

Development Tools

- Visual Studio Code
- Git
- GitHub
- Postman

8. Module Description

Module 1: User Authentication

Purpose

Provide secure access to the application.

Functions

- User registration
- User login
- Session management
- Access control

Benefits

- Secure system access
- User accountability
- Protection against unauthorized access

Module 2: Detection Engine

Purpose

Identify sensitive information within source code files.

Functions

- Pattern matching
- Secret detection
- Risk categorization
- Alert generation

Detectable Information

- API Keys
- Passwords
- JWT Tokens
- Access Tokens
- Email Addresses
- Database Credentials

Module 3: VS Code Extension

Purpose

Provide real-time monitoring during software development.

Functions

- Continuous file scanning
- Real-time notifications
- Security warnings
- Developer guidance

Benefits

- Early detection
- Improved developer awareness
- Reduced leakage risks

Module 4: Incident Management

Purpose

Track and manage detected incidents.

Functions

- Incident recording
- Status management
- Investigation support
- Historical tracking

Benefits

- Improved visibility
- Incident accountability
- Easier investigation process

Module 5: Analytics Dashboard

Purpose

Provide centralized monitoring and reporting.

Functions

- Security statistics
- Incident visualization
- Trend analysis
- Report generation

Benefits

- Better decision-making
- Security monitoring
- Threat analysis

9. Database Design

Users Collection

Field	Data Type
userId	String
name	String
email	String
password	String
role	String

Incidents Collection

Field	Data Type
incidentId	String
leakType	String
severity	String

description	String
timestamp	Date
status	String

Audit Logs Collection

Field	Data Type
logId	String
userId	String
activity	String
timestamp	Date

10. Workflow

Step 1

Developer writes source code in Visual Studio Code.

Step 2

VS Code extension scans files for sensitive patterns.

Step 3

Detection engine identifies potential leaks.

Step 4

Warning notification is displayed.

Step 5

Incident details are sent to backend APIs.

Step 6

Data is stored in MongoDB.

Step 7

Dashboard displays incidents and analytics.

Step 8

Administrator reviews and manages alerts.

11. Security Features

The system incorporates several security mechanisms:

- User Authentication
- Access Control
- Incident Logging
- Real-Time Monitoring
- Secret Detection
- Security Alerts
- Audit Trails
- Centralized Visibility

These features collectively reduce the likelihood of accidental or intentional data exposure.

12. Testing

Functional Testing

Test Case	Expected Result
Login	User authenticated
Detection	Sensitive data identified
Alert Generation	Notification displayed
Incident Logging	Data stored successfully

Security Testing

- Authentication testing
- Authorization testing
- Input validation

- Session management verification

Performance Testing

- Detection speed evaluation
- Dashboard responsiveness
- Database query performance

13. Results

The developed system successfully:

- Detected sensitive information in source code.
- Generated real-time security alerts.
- Logged incidents for future analysis.
- Provided centralized visibility through a web dashboard.
- Improved developer awareness regarding secure coding practices.

The integration of the VS Code extension and web application created a complete ecosystem for proactive data leakage prevention.

14. Advantages

- Real-time monitoring
- Easy integration with development workflow
- Improved security awareness
- Centralized incident management
- Scalable architecture
- Reduced security risks

15. Limitations

- Pattern-based detection may generate false positives.
- Limited detection of highly obfuscated secrets.
- Requires rule updates for emerging secret formats.
- Dependency on predefined patterns.

16. Future Enhancements

Future versions of the system can include:

- Artificial Intelligence-based detection
- Machine Learning risk prediction
- GitHub repository scanning
- Cloud deployment
- Email and Slack notifications
- OCR-based document scanning
- Compliance reporting (GDPR, HIPAA, ISO 27001)
- Automated secret revocation

17. Applications

The system can be used in:

- Software Development Companies
- Financial Institutions
- Healthcare Organizations
- Government Agencies
- Educational Institutions
- Startups
- Cloud Development Teams

18. Conclusion

The Data Leakage Prevention and Detection System provides an effective approach to identifying and preventing sensitive information exposure during software development. By integrating a VS Code extension for real-time detection with a centralized web dashboard for monitoring and incident management, the system helps organizations reduce security risks and improve secure development practices.

The project demonstrates practical implementation of cybersecurity principles, secure software development methodologies, and full-stack application development. It provides a scalable foundation for future enhancements involving artificial intelligence, advanced analytics, and enterprise security integrations.